

Complements of Operational Research

3.1 Sequencing problems: Johnson's rule (2 machines)

José Firmino Aguilar Madeira

Why sequence?

A **print shop** receives jobs that all go through the same two machines, **in the same order**: first **printing**, then **finishing** (cutting, folding, binding).

In what order should the jobs be done?

Goal: minimise the **total time** (*makespan*) T — the instant at which the last job leaves finishing. Smaller $T \Rightarrow$ less idle time, less overtime, earlier deliveries.

The same problem appears in workshops (machining $\rightarrow$ assembly), laboratories (preparation $\rightarrow$ analysis) and *data centres* (read $\rightarrow$ process).

Assumptions

- processing times are known;
- each machine does one job at a time;
- a job, once started, is not interrupted;
- all jobs are available at instant 0;
- the same sequence on both machines.

How many sequences? $n!$ $6! = 720$,
 $10! \approx 3.6 \times 10^6$, $20! \approx 2.4 \times 10^{18}$.

The example of this chapter: six jobs in the print shop

Job		Printing (h)	Finishing (h)
A	Catalogues	8	9
B	Magazines	11	5
C	Business cards	5	2
D	Invitations	3	5
E	Books	4	12
F	Posters	10	9

Sum: 41 h of printing and 42 h of finishing.

First idea: do them in order of arrival, A, B, C, D, E, F.

The example of this chapter: six jobs in the print shop

Job		Printing (h)	Finishing (h)
A	Catalogues	8	9
B	Magazines	11	5
C	Business cards	5	2
D	Invitations	3	5
E	Books	4	12
F	Posters	10	9

Sum: 41 h of printing and 42 h of finishing.

First idea: do them in order of arrival, A, B, C, D, E, F.

How is T computed? For each job, in order:

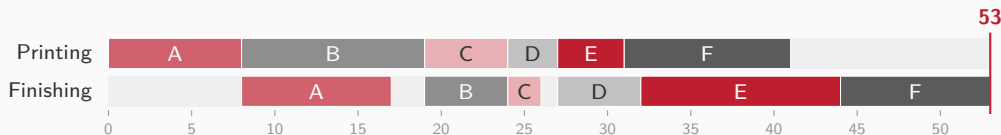
$$C_1 \leftarrow C_1 + a_j, \quad C_2 \leftarrow \max(C_2, C_1) + b_j$$

(a_j, b_j : times in printing and in finishing; C_1, C_2 : instants at which each machine becomes free).

Finishing only starts a job when it has left printing **and** the previous one has left finishing.

In order of arrival: $T = 53$ h

Job	A	B	C	D	E	F
Leaves printing (C_1)	8	19	24	27	31	41
Leaves finishing (C_2)	17	24	26	32	44	53



Finishing is left **waiting** at the start (8 h, while A is printed) and between jobs (B leaves at 19 h, but finishing became free at 17 h). In total, $53 - 42 = 11$ h of finishing standing still; printing ends at 41 h and stands idle for 12 h until the end.

Question: is there a better order? Which is the best of the 720?

Johnson's rule (1954)

Algorithm (2 machines)

1. Find the **smallest** of all the times not yet assigned.
2. If it is on **machine 1**, the job goes to the **first** free position.
If it is on **machine 2**, it goes to the **last** free position.
3. Cross out that job and go back to 1.

Ties: any choice gives an optimal sequence.

Equivalent form: first the jobs with $a_j \leq b_j$, in increasing order of a_j ; then the jobs with $a_j > b_j$, in decreasing order of b_j .

Intuition

- **Start** with a short printing: finishing starts early and is not left waiting.
- **End** with a short finishing: once printing is over, little is left to do.

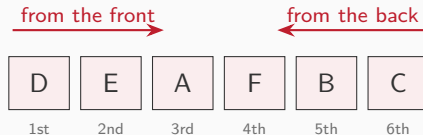
Result

Johnson's rule always gives an **optimal** sequence (with 2 machines).

Johnson step by step

Step	Smallest time	Machine	Job	Position
1	2	finishing	C	6th (last)
2	3	printing	D	1st
3	4	printing	E	2nd
4	5	finishing	B	5th
5	8	printing	A	3rd
6	—	—	F	4th

In step 4, D's 5 in finishing no longer counts (D has already been assigned): the smallest not yet assigned is B's 5. In step 5, A (8, 9) and F (10, 9) remain: the smallest is A's 8, in printing; F takes the only free position.

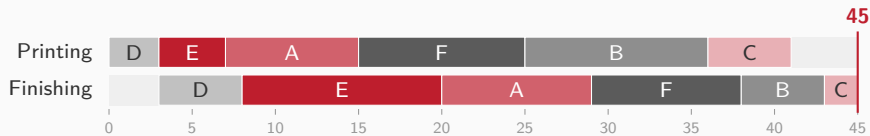


Johnson sequence: D, E, A, F, B, C.

By the equivalent form: $a \leq b$: D (3), E (4), A (8) by increasing a ; $a > b$: F (9), B (5), C (2) by decreasing b .

With Johnson: $T = 45$ h (8 h less)

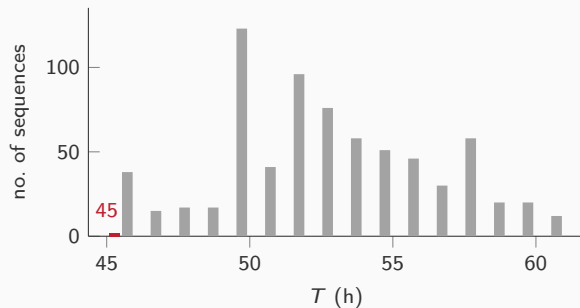
Job	D	E	A	F	B	C
Leaves printing (C_1)	3	7	15	25	36	41
Leaves finishing (C_2)	8	20	29	38	43	45



Finishing only waits 3 h (at the start) and never stops again: $45 - 42 = 3$ h idle. Printing ends at 41 h and is idle for 4 h.

Gain: from 53 h to 45 h, -15% , without buying machines or hiring anyone — just by changing the order.

Is Johnson really optimal? Enumerating the 720 sequences



- The minimum is 45 h: only 2 of the 720 sequences reach it (D E A F B C and D E F A B C).
- The mean is 53 h — the order of arrival (53 h) is a “typical” sequence.
- The worst gives 61 h.
- The two optimal ones differ only in the order of A and F; Johnson finds one of them.

With 6 jobs we can still enumerate; with 20 we no longer can. Johnson solves it in n steps, whatever n is.

```
def johnson(a, b):  
    """a, b: dictionaries job -> time."""  
    L = [j for j in a if a[j] <= b[j]]  
    R = [j for j in a if a[j] > b[j]]  
    L.sort(key=lambda j: a[j])    # increasing a  
    R.sort(key=lambda j: -b[j])   # decreasing b  
    return L + R
```

```
def tempo_total(seq, a, b):  
    c1 = c2 = 0  
    for j in seq:  
        c1 += a[j]  
        c2 = max(c2, c1) + b[j]  
    return c2
```

```
a = dict(A=8, B=11, C=5, D=3, E=4, F=10)  
b = dict(A=9, B=5, C=2, D=5, E=12, F=9)  
s = johnson(a, b)  
print(s, tempo_total(s, a, b))  
# ['D', 'E', 'A', 'F', 'B', 'C'] 45
```

Check by brute force:

```
from itertools import permutations  
min(tempo_total(p, a, b)  
    for p in permutations(a))  
# 45
```

Full file (Johnson for two machines and brute force):

code/python/03_scheduling/03_1_johnson/
sequenciamento.py

Example 1 — nine jobs, two machines

Job	J1	J2	J3	J4	J5	J6	J7	J8	J9
M1 (h)	2	5	4	9	6	8	7	5	4
M2 (h)	6	8	7	4	3	9	3	8	11

Determine the optimal sequence, the total time and the idle time of each machine. (Comes back in 3.3, with the TDM.)

Example 2 — seven jobs, two machines

Job	1	2	3	4	5	6	7
M1 (h)	3	12	15	6	10	11	9
M2 (h)	8	10	10	6	12	1	3

Determine the optimal sequence, the total time and the idle time of each machine. Are there ties? Do they change the total time?

KEY TAKEAWAYS — 3.1

1. Problem: n jobs, 2 machines, same order; minimise the total time T . There are $n!$ sequences.
2. Completion times: $C_1 \leftarrow C_1 + a_j$, $C_2 \leftarrow \max(C_2, C_1) + b_j$. Idle time of each machine $= T - \text{sum of its times}$.
3. **Johnson**: the smallest time not yet assigned; on machine 1 $\rightarrow$ first free place; on machine 2 $\rightarrow$ last free place.
4. With 2 machines, Johnson is **optimal**. In the example: 53 h $\rightarrow$ 45 h just by changing the order.
5. Always check: recompute T for the sequence obtained (and, with few jobs, compare with brute force).

Next: 3.2 Three or more machines — when does Johnson still work?

References and resources

References Full list: *Course bibliography*.

- Johnson, S. M., Optimal two- and three-stage production schedules with setup times included, *Naval Research Logistics Quarterly*, 1(1), 61–68, 1954.
- Kasana, H. S., Kumar, K. D., *Introductory Operations Research*, Springer, 2004 — ch. 10, Sequencing Problems.
- Taha, H. A., *Operations Research: An Introduction*, 10th ed., Pearson, 2017.
- Pinedo, M. L., *Scheduling: Theory, Algorithms, and Systems*, 5th ed., Springer, 2016 — *flow shops*.

Resources

- Exercise sheet 3. `code/python/03_scheduling/03_1_johnson/sequenciamento.py`.